

Nottsvale Floor Plan Code App Build Guide

This is my step by step for the floor plan code app, shot 9.9 of the Nottsvale series. I worked through this in a dry run before recording a clean pass against it.

I checked everything here against Microsoft Learn as at 28 August 2026: the code apps CLI reference, the npm quickstart, the Connect to Dataverse how to, the SDK v1.0 migration guide, and the code apps overview.

Updated 18 September 2026 with two corrections that only came up once I'd actually built the thing: the static asset path in steps 4 and 5, and a note on null versus undefined below. See my bug log document for the full list of eleven bugs I hit building this specific app, with the root cause and fix for each. The null versus undefined boundary in particular caught me twice, so it's worth reading before writing the write path code in step 8.

What I'm building, and why it's a code app

An interactive site floor plan. I pick a Nottsvale site, see its equipment plotted on a scale drawing, colour coded by how many maintenance requests are open against it, and I can click a machine to see its request history.

The justification matters more than the build itself. Every other "why a code app" argument I could make reduces to canvas would be awkward. This one doesn't. Power Fx cannot render an interactive scale drawing with clickable hit regions. There's no low code route to it at all, and that single fact is the strongest thing in the video. It's why I chose this over a scheduling board or a data grid.

I made the app read only by design. Writing a lookup column from a code app is currently awkward. Microsoft's own guidance says it needs associating with a single valued navigation property or associating on create, and a dedicated guide for it is still pending. A view that writes nothing sidesteps that entirely, and "this is a view, not a form" is a stronger position for me to take than half implementing CRUD in React.

Prerequisites

Environment

Code apps have to be enabled on the environment by an admin. This isn't on by default. Without it, `pa app push` fails with a "does not allow this operation for this Code app" error right at the end of a session, which is annoying to discover too late. It's shot 3.10 in my shot list.

End users need a Power Apps Premium licence. My Developer Plan environment covers me for building.

I needed the environment ID, which I got from the Power Platform admin centre, or with `pa env list`.

Tooling

```
npm install --global @microsoft/power-apps-cli@latest
npm install --global @microsoft/power-apps@latest
```

Current versions when I did this: CLI 1.0.0, SDK 1.3.0. The npm based `pa` CLI went generally available alongside SDK 1.3.0 and is now the primary experience. I confirmed with `pa --version` before recording.

Two things caught me out from older material I found while researching this:

The legacy `power-apps` command has been removed. If a tutorial uses it, that tutorial is out of date.

`pac code` commands still work but aren't recommended any more. Short aliases are gone, and `IS_PAC_CLI` and `PAC_CLI_TELEMETRY_OPTOUT` aren't used any more either.

Don't call initialize()

SDK 1.0 and later removed the `initialize` function entirely. You make data calls, retrieve context and interact with the platform directly, with nothing to wait on.

Almost every blog post written before spring 2026 shows this pattern. It's dead code:

```
1 // Don't do this. initialize() no longer exists.
2 import { initialize } from '@microsoft/power-apps/app';
3 const [isInitialized, setIsInitialized] = useState(false);
4 useEffect(() => { async () => { await initialize(); setIsInitialized(true); }}(); }, []);
```

If the Vite template ships a `PowerProvider.tsx` that calls `initialize`, that file is stale. I checked it and removed the call.

Data

I needed three Dataverse tables populated: Site, Asset (including Plan X and Plan Y), and Maintenance Request. My CSVs cover this. I deliberately weighted the request data so four assets carry three or more open requests and read red, eight carry one or two and read amber, and the rest read green. Without that weighting the payoff shot is just a plan covered in identical dots.

Step 1: Scaffold the project

```
npx degit github:microsoft/PowerAppsCodeApps/templates/vite nottsvale-floorplan
cd nottsvale-floorplan
npm install
```

This is an ordinary Vite, React and TypeScript project. Nothing about it is Power Apps specific yet, worth pointing out on camera, because it's what makes code apps feel different from PCF or canvas.

Step 2: Link the project to my environment

```
pa auth login
pa auth status
pa app init
```

I signed in first so the browser flow didn't interrupt the next command. `pa app init` prompted for the environment ID and a display name, then wrote `power.config.json` into the project root. That file holds the app identity and, shortly, its data sources. It's the join between a normal front end project and the platform.

Both prompts can be passed directly:

```
pa app init --display-name "Nottsvale Floor Plan" --environment-id <environment-id>
```

I ran it bare on camera. The two prompts are more instructive than the one liner.

Step 3: Add the Dataverse tables as data sources

Dataverse is special cased. The connector value is the literal word `dataverse`, and no connection or connection ID is required.

```
pa app add data-source --connector dataverse --table nvm_site
pa app add data-source --connector dataverse --table nvm_asset
pa app add data-source --connector dataverse --table nvm_maintenancerequest
```

`--table` takes the logical name, not the display name. For my solution that's `nvm_` plus the singular table name, lower case.

If I add a connector data source later (Office 365 Users, SQL, SharePoint) those do need a connection first, and the flow is different:

```
pa connector list --search <term>
pa connection create --connector <connector-id>
pa connection list
pa app add data-source --connector <connector-id> --connection-id <connection-id>
```

Full parameter set: `--connector`, `--connection-id`, `--connection-ref`, `--solution-id`, `--dataset`, `--table`, `--procedure`.

For anything going through ALM, I'd use `--connection-ref` with `--solution-id` rather than a direct connection ID. A direct connection is user specific and environment bound. A connection reference is a parameter supplied at import time. My Video 9 narration claims the app moves between environments with the solution, and that's only true on the connection reference form.

Read the generated filenames before writing any code

Adding a data source generates a model and a service under `src/generated/`. The names follow the entity set, plural and capitalised, not the logical name. Microsoft's documented example is that the `account` table produces `AccountsModel.ts` and `AccountsService.ts`.

So `nvm_site` will most likely give me `Nvm_sitesService`, but I don't assume that. I open `src/generated/services` and `src/generated/models`, read the real filenames and exported symbols, and use those. Every import below is a placeholder I have to replace.

This is worth filming. Opening the generated folder and reading what's actually there is an honest step and it sets up the next beat.

What the generated files buy me

Strongly typed models built from my real table schema, my columns, my choice values, my relationships. IntelliSense completes column names that came out of Dataverse, and a misspelling is a compile error rather than a blank screen a user reports six weeks later.

Available on every generated Dataverse service: `create()`, `get()`, `getAll()`, `update()`, `delete()`, and `getMetadata()`.

If I change a table schema afterward, I regenerate with:

```
pa app refresh data-source --name <data-source-name>
```

Step 4: Add the plan drawings

I put the three SVGs under `src/`, not `public/`:

```
src/plans/warrington-works.svg
src/plans/wakefield-plant.svg
src/plans/wigan-site.svg
```

This is a correction on my part, not a preference. An earlier version of my own notes put the SVGs in `public/plans/` and referenced them by path string. That works fine in Local Play and is wrong once published. A published code app’s asset routing doesn’t serve `public/` relative paths the same way a generic static host does, and the request fails with `RouteNotFound`, not a plain 404. I found this the hard way, running against Local Play, a static `npmx serve dist` test, and finally the actual published app before the real cause showed up in the Network tab.

`public/` assets are referenced by path at runtime. Assets under `src/`, imported as ES modules, are resolved by the bundler at build time. Vite emits them into `dist/assets/` with a content hashed filename and gives back the correct URL for wherever the build actually ends up being served. That’s the difference that matters here, and it’s why step 5 below imports rather than references by string.

All three share a 1000 by 600 viewBox, and every equipment footprint on the drawings sits at the same Plan X and Plan Y as the matching Asset row. A marker lands on the machine, not on an empty rectangle.

Step 5: Map sites to drawings

`src/planMap.ts`:

```
1 import warringtonSvg from './plans/warrington-works.svg';
2 import wakefieldSvg from './plans/wakefield-plant.svg';
3 import wiganSvg from './plans/wigan-site.svg';
4
5 export const PLAN_BY_SITE: Record<string, string> = {
6   'Warrington Works': warringtonSvg,
7   'Wakefield Plant': wakefieldSvg,
8   'Wigan Site': wiganSvg,
9 };
```

This is the one hardcoded thing in the app. The drawings are static assets, so something has to associate a site row with a file. Everything else comes from Dataverse.

I own that on camera rather than glossing over it. The production answer is a `Plan File` text column on the Site table, read like any other column. Worth saying, because it shows I know the difference between a demo shortcut and a real design.

I wouldn’t reach for `import.meta.env.BASE_URL` here as an alternative fix if a path string version ever breaks again. It looks like the standard Vite answer for base path issues, and it does nothing for this specific problem. The asset import approach above is the actual fix, confirmed working in the published environment, not a theory.

Step 6: Load the data

`src/useFloorPlanData.ts`. One hook, three queries, derived open request counts.

Always pass select

Microsoft’s guidance is explicit: always limit the number of columns retrieved using the `select` parameter. `getAll` takes an optional options object:

```
1 interface IGetAllOptions {
2   maxPageSize?: number;
3   select?: string[];
4   filter?: string;
5   orderBy?: string[]; // capital B
6   top?: number;
7   skip?: number;
8   skipToken?: string;
9 }
```

`result.data` is the array itself, not `result.data.value`.

```
1 import { useEffect, useState } from 'react';
2 // replace all three with the real generated names from src/generated/services
3 import { SitesService } from './generated/services/SitesService';
4 import { AssetsService } from './generated/services/AssetsService';
5 import { RequestsService } from './generated/services/RequestsService';
6
7 export type Site = { id: string; name: string };
8 export type Asset = {
9   id: string; name: string; tag: string; siteId: string;
10   criticality?: string; planX: number; planY: number; openCount: number;
11 };
```

```

12 export type Request = {
13   id: string; reference: string; assetId: string;
14   status?: string; priority?: string; reportedOn?: string; downtime?: number;
15 };
16
17 // fallback if formatted values aren't present alongside select
18 const STATUS_LABEL: Record<number, string> = {
19   1: 'New', 2: 'Triaged', 3: 'Assigned',
20   4: 'In Progress', 5: 'Complete', 6: 'Cancelled',
21 };
22 const OPEN = new Set(['New', 'Triaged', 'Assigned', 'In Progress']);
23 const FV = '@0Data.Community.Display.V1.FormattedValue';
24
25 export function useFloorPlanData() {
26   const [sites, setSites] = useState<Site[]>([]);
27   const [assets, setAssets] = useState<Asset[]>([]);
28   const [requests, setRequests] = useState<Request[]>([]);
29   const [loading, setLoading] = useState(true);
30   const [error, setError] = useState<string | null>(null);
31
32   useEffect(() => {
33     (async () => {
34       try {
35         const [s, a, r] = await Promise.all([
36           SitesService.getAll({
37             select: ['nvm_siteid', 'nvm_sitename'],
38             orderBy: ['nvm_sitename asc'],
39           }),
40           AssetsService.getAll({
41             select: ['nvm_assetid', 'nvm_assetname', 'nvm_assettag',
42               'nvm_site_value', 'nvm_criticality',
43               'nvm_planx', 'nvm_plany'],
44           }),
45           RequestsService.getAll({
46             select: ['nvm_maintenancerequestid', 'nvm_requestreference',
47               'nvm_asset_value', 'nvm_status', 'nvm_priority',
48               'nvm_reportedon', 'nvm_downtimehours'],
49             orderBy: ['nvm_reportedon desc'],
50           }),
51         ]);
52
53         const siteRows: Site[] = (s.data ?? []).map((x: any) => ({
54           id: x.nvm_siteid,
55           name: x.nvm_sitename,
56         }));
57
58         const reqRows: Request[] = (r.data ?? []).map((x: any) => ({
59           id: x.nvm_maintenancerequestid,
60           reference: x.nvm_requestreference,
61           assetId: x._nvm_asset_value,
62           status: x[`nvm_status${FV}`] ?? STATUS_LABEL[x.nvm_status],
63           priority: x[`nvm_priority${FV}`],
64           reportedOn: x.nvm_reportedon,
65           downtime: x.nvm_downtimehours,
66         }));
67
68         const openByAsset = new Map<string, number>();
69         for (const req of reqRows) {
70           if (req.status && OPEN.has(req.status)) {
71             openByAsset.set(req.assetId, (openByAsset.get(req.assetId) ?? 0) + 1);
72           }
73         }
74
75         const assetRows: Asset[] = (a.data ?? []).map((x: any) => ({
76           id: x.nvm_assetid,
77           name: x.nvm_assetname,
78           tag: x.nvm_assettag,
79           siteId: x._nvm_site_value,
80           criticality: x[`nvm_criticality${FV}`],
81           planX: x.nvm_planx ?? 50,
82           planY: x.nvm_plany ?? 50,
83           openCount: openByAsset.get(x.nvm_assetid) ?? 0,
84         }));
85
86         setSites(siteRows);
87         setAssets(assetRows);
88         setRequests(reqRows);
89       } catch (e: any) {
90         setError(e?.message ?? 'Failed to load plant data');
91       } finally {
92         setLoading(false);
93       }
94     })();
95   }, []);
96
97   return { sites, assets, requests, loading, error };
98 }

```

Four things I want to hold on during the montage.

Lookups read as `_fieldname_value`. The Asset table's Site lookup arrives as `_nvm_site_value` holding a GUID, not a nested object. That's OData convention.

Lookups are easy to read, awkward to write. Microsoft states that writing one currently requires associating with a single valued navigation property or associating on create, with a dedicated guide still coming soon. My app writes nothing, which is exactly why it dodges the problem. One line on camera.

Formatted values give me choice labels. This is a documented supported scenario. I verified in my dry run that they arrive alongside `select`. If they hadn't, the `STATUS_LABEL` fallback would cover it.

`Promise.all`, not three sequential awaits. Three round trips in parallel rather than in series. Visibly faster, and a reasonable thing to point at.

I read the real column names out of `src/generated/models` rather than trusting the `nvm_` names above.

Step 7: The plan component

`src/FloorPlan.tsx`. The SVG is the background layer, markers positioned over it in percentages.

```
1 import type { Asset } from './useFloorPlanData';
2
3 type Props = {
4   planUrl: string;
5   assets: Asset[];
6   selectedId: string | null;
7   onSelect: (id: string) => void;
8 };
9
10 function colourFor(openCount: number) {
11   if (openCount === 0) return { fill: '#16A34A', ring: '#DCFCF7' };
12   if (openCount <= 2) return { fill: '#F59E0B', ring: '#FEF3C7' };
13   return { fill: '#DC2626', ring: '#FEE2E2' };
14 }
15
16 export function FloorPlan({ planUrl, assets, selectedId, onSelect }: Props) {
17   return (
18     <div style={{ position: 'relative', width: '100%', aspectRatio: '1000 / 600' }}>
19       <img src={planUrl} alt="" style={{ width: '100%', height: '100%', display: 'block' }} />
20
21       {assets.map((a) => {
22         const c = colourFor(a.openCount);
23         const on = a.id === selectedId;
24         return (
25           <button
26             key={a.id}
27             onClick={() => onSelect(a.id)}
28             aria-label={` ${a.name}, ${a.openCount} open requests`}
29             title={` ${a.name} open ${a.openCount}`}
30             style={{
31               position: 'absolute',
32               left: `${a.planX}%`,
33               top: `${a.planY}%`,
34               transform: 'translate(-50%, -50%)',
35               width: on ? 28 : 20,
36               height: on ? 28 : 20,
37               borderRadius: '50%',
38               background: c.fill,
39               border: `3px solid ${on ? '#1F2937' : c.ring}`,
40               cursor: 'pointer',
41               padding: 0,
42               transition: 'width .12s, height .12s',
43             }}
44           />
45         );
46       })}
47     </div>
48   );
49 }
```

Why percentages over an `<img>`. The plan scales to any container width and markers stay put. No viewBox maths, no coordinate transforms, no resize listener. It also keeps the drawing a swappable asset. Replace the SVG and nothing in the component changes.

I use `<button>`, not `<div>`. Keyboard focus and screen reader support for free, and `aria-label` makes each marker meaningful. A good answer if accessibility comes up in an interview.

Step 8: Site selector and detail panel

```

1 import { useMemo, useState } from 'react';
2 import { useFloorPlanData } from './useFloorPlanData';
3 import { PLAN_BY_SITE } from './planMap';
4 import { FloorPlan } from './FloorPlan';
5
6 export default function App() {
7   const { sites, assets, requests, loading, error } = useFloorPlanData();
8   const [siteId, setSiteId] = useState<string | null>(null);
9   const [assetId, setAssetId] = useState<string | null>(null);
10
11   const activeSite = sites.find(s => s.id === siteId) ?? sites[0];
12   const planUrl = activeSite ? PLAN_BY_SITE[activeSite.name] : undefined;
13
14   const siteAssets = useMemo(
15     () => assets.filter(a => a.siteId === activeSite?.id),
16     [assets, activeSite]
17   );
18
19   const selected = siteAssets.find(a => a.id === assetId) ?? null;
20   const selectedRequests = useMemo(
21     () => requests.filter(r => r.assetId === assetId),
22     [requests, assetId]
23   );
24
25   if (loading) return <p>Loading plant data...</p>;
26   if (error) return <p role="alert">{error}</p>;
27
28   return (
29     <div style={{ display: 'grid', gridTemplateColumns: '1fr 320px', gap: 24, padding: 24 }}>
30       <div>
31         <div style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
32           {sites.map(s => (
33             <button
34               key={s.id}
35               onClick={() => { setSiteId(s.id); setAssetId(null); }}
36               style={{
37                 padding: '8px 14px',
38                 fontWeight: s.id === activeSite?.id ? 700 : 400,
39                 background: s.id === activeSite?.id ? '#1F2937' : '#F3F4F6',
40                 color: s.id === activeSite?.id ? '#fff' : '#111',
41                 border: 'none', borderRadius: 6, cursor: 'pointer',
42               }}
43             >
44               {s.name}
45             </button>
46           ))}
47         </div>
48
49         {planUrl && (
50           <FloorPlan
51             planUrl={planUrl}
52             assets={siteAssets}
53             selectedId={assetId}
54             onSelect={setAssetId}
55           />
56         )}
57       </div>
58
59       <aside>
60         {!selected && <p style={{ color: '#6B7280' }}>Select an asset on the plan.</p>}
61         {selected && (
62           <div>
63             <h2 style={{ margin: '0 0 4px' }}>{selected.name}</h2>
64             <p style={{ margin: '0 0 16px', color: '#6B7280' }}>
65               {selected.tag}, {selected.criticality}
66             </p>
67             <h3 style={{ fontSize: 14, textTransform: 'uppercase', letterSpacing: 1 }}>
68               Open requests ({selected.openCount})
69             </h3>
70             {selectedRequests.length === 0 && <p>No requests on record.</p>}
71             <ul style={{ listStyle: 'none', padding: 0 }}>
72               {selectedRequests.map(r => (
73                 <li key={r.id} style={{ padding: '10px 0', borderTop: '1px solid #E5E7EB' }}>
74                   <strong>{r.reference}</strong> {r.status}
75                   <div style={{ color: '#6B7280', fontSize: 13 }}>
76                     {r.priority}, {r.downtime ?? 0} h downtime
77                   </div>
78                 </li>
79               ))}
80             </ul>
81           </div>
82         )}
83       </aside>
84     </div>
85   );
86 }

```

Selecting a site clears the asset selection. Small thing, but without it I get a detail panel showing an asset that isn't on the visible plan.

Step 9: Run it locally

```
pa app run
```

Two URLs print. I open the one labelled Local Play, in my tenant browser profile.

Local Play loads my local build inside the Power Apps host, so I'm testing against the real runtime, real authentication and real connectors rather than a mock. Local dev runs on port 3000, not Vite's usual 5173.

I sort the browser permission before recording. Chrome and Edge block requests from public origins to local endpoints by default, and my app talks to localhost. I may need to grant permission when prompted. For embedded scenarios I'd add `allow="local-network-access"` to the iframe.

Step 10: Build and publish

```
npm run build
pa app push
```

The build has to run first. Push uploads compiled output, not source. On success it returns the Power Apps URL where the app runs.

To put the app inside my solution so it travels with everything else:

```
pa app push --solution-id <solution-id>
```

I find solutions with `pa solution list --search nottsvale`.

For automated publishing there's `pa app push --non-interactive`, which authenticates as a service principal instead of a human. I don't need this for the series. See my appendix below for what it would involve and why it's a later project.

Step 11: Check it before recording

I run through this against the Local Play URL. Anything found here is a fix. Anything found on camera is a reshoot.

No `initialize` anywhere in the project, checked with a grep.

`pa --version` reports 1.0.0 or later, and `npm ls @microsoft/power-apps` inside the project reports 1.3.x.

Dev server on port 3000, no console errors on load.

All three site buttons appear, named from Dataverse.

Clicking a site swaps the plan and the markers together.

Every marker lands on its equipment footprint, not a wall or the yard.

Four assets read red and several read green, so the colour coding is obvious.

Clicking a marker fills the side panel with real request rows.

Status and priority show as labels, not integers.

Selecting a new site clears the previous selection.

Resizing the window keeps markers on their footprints.

What I actually filmed in the montage

Shot 9.9 is jump cuts. I picked moments rather than filming everything.

`planMap.ts` being written. Three lines, sets up the one hardcoded thing beat.

Opening `src/generated/services` to read the real service names.

The `select` arrays, with IntelliSense completing column names off the generated model.

The `_nvm_site_value` lookup line, held for a beat, a genuine gotcha.

The percentage positioning in `FloorPlan.tsx`.

First render with markers in the wrong place, then corrected.

I landed on the finished component and cut to the next shot.

Things worth saying on camera

Code apps are GA

They went generally available on 5 February 2026. Not preview, not experimental. What has moved fast is the tooling, the CLI has been replaced twice this year and the SDK had a breaking major version, which is a more interesting and more defensible point to make. I put the recording date on screen.

The documentation is currently split

Microsoft's own pages disagree with each other. The quickstart and CLI reference use `pa`, some how to pages still show `pac code`, and at least one states that no data source refresh command exists, which the current reference contradicts. I named that on camera. It's credibility, not criticism, and it explains why viewers will find conflicting instructions everywhere they look.

Delegation, as a callback to the canvas video

Code apps support delegation for `filter`, `sort` and `top`, plus paging. Same platform principle as the delegation warning in the canvas app shot. The operation is evaluated at the data source, not in the client. Canvas gives you a warning triangle. Here I get a typed options object and I'm expected to know why it matters.

Don't bring PCF habits

No `context.webAPI`, no `context.parameters`, no `Xrm.WebApi`. Code apps use generated service classes and `getContext()`. Worth one line for anyone arriving from PCF.

The honest limits

Not supported in Dataverse: polymorphic lookups, entity metadata CRUD, FetchXML, alternate keys. None of these affect my app.

Platform limits: no Power Apps mobile support, iOS or Android. Microsoft's own response on the feature request is that the mobile player isn't supported and sits in the backlog with no ETA.

No Power Apps for Windows.

Standalone only. Can't be embedded in a canvas or model driven app. They can be embedded in a Power BI report via the Power Apps visual.

No `PowerBIIntegration` function, no SharePoint forms integration, no Power Platform Git integration.

The mobile limit is a narration beat, not a footnote. In my model driven video I show the app running unchanged on a phone. Here I have to say the code app doesn't run on the mobile player at all. That's the trade. I buy capability the low code approach can't reach, and I pay for it in reach.

One governance caveat

My closing argument is that the app inherits Entra authentication, DLP and conditional access. All true. But code apps don't use Storage SAS IP restriction, so the SAS IP Binding and Firewall environment setting doesn't apply. The compiled assets are served from a publicly accessible endpoint.

One sentence on that makes the governance argument credible rather than promotional, and a platform architect watching will already know it. Better I get there first.

Optional extras if I have time

A colour legend in the corner. Green none, amber one to two, red three or more. Ten minutes of work, and the payoff shot reads instantly without narration explaining it.

`appUrl` on `IAppContext` lets the app read its own running URL. Useful for a copy link to this asset button, and it's a recent addition worth mentioning.

Power Automate flows can be called from a code app with `pa app add flow --flow-id <id>`, discovered with `pa app list-flows`. Only solution aware instant flows using the Power Apps trigger are supported. If I ever wanted the plan to raise a request without writing a lookup from React, this is the route, the flow does the write server side.

Appendix: CI/CD, for another time

I'm not building this for the series. It's an Entra app registration, credential handling and a workflow file, all to automate a two command deploy I'll run maybe fifteen times. It would add a recording session and would demonstrate GitHub Actions rather than the Power Platform. One sentence in Video 9 saying the capability exists is the right amount.

Written down here so it's captured while it's fresh.

What CI/CD actually means

Continuous integration means every push triggers an automated build and test run, so a break surfaces in minutes rather than at release.

Continuous deployment means once the build passes, it deploys without anyone clicking anything.

For this app that means instead of running `npm run build` and `pa app push` from my laptop, I'd push to GitHub and a workflow would run those same commands on a build server.

-non-interactive is one small piece, not the thing itself

The flag doesn't create CI/CD. It changes where the CLI looks for credentials. Instead of the cached session from `pa auth login`, it reads service principal credentials from environment variables and authenticates as that identity. No browser is the consequence, not the purpose.

On its own it just fails differently, an auth error instead of a browser window.

The four pieces I'd actually need

A workflow file, `.github/workflows/deploy.yml`, defining what runs and when.

A service principal, an app registration in Entra, giving the build server an identity.

The app shared with it: `pa app share --principal <enterprise-application-object-id> --access edit`, using the Enterprise Application object ID, not the App Registration object ID.

Credentials available to the job: `PA_CLI_USE_SP_AUTH`, `SP_CLIENT_ID` and the rest.

The workflow then runs `npm install`, `npm run build`, `pa app push --non-interactive`.

No automatic testing comes for free. CI only runs tests if I've written tests and added a step to run them.

Deploying to a solution is a separate question

Two things that are easy to conflate. `pa app push --solution-id <id>` puts the app into my solution in the development environment. That's what makes it appear inside `Nottsvale Maintenance` in the ALM shot.

Getting it to test or production is still a managed solution export and import, or a pipeline run, exactly what my ALM video covers.

CI/CD handles the code. ALM handles the solution. They're separate tracks that meet at the dev environment. Whether `--solution-id` and `--non-interactive` combine on one command is worth confirming rather than assuming.

On secrets, since this is where people get hurt

Committing a secret to the repo is unsafe. A client secret in a `.env` file, a workflow file, or anywhere in the source tree is exposed permanently. It stays in git history after deletion, and bots scan public repos within minutes of a push.

GitHub Secrets is a different mechanism and is reasonable. Stored encrypted, injected into the job at runtime, masked in logs so a leaked value prints as asterisks. I can't read one back through the UI, only overwrite it.

The residual risks are real but bounded. Anyone who can modify a workflow file can write one that exfiltrates a secret, so repo write access effectively equals secret access. A compromised npm dependency running in the build can read the environment.

Better than storing a secret at all is OpenID Connect. GitHub Actions can authenticate to Entra with a short lived token issued per job, with no stored credential. I'd configure a federated credential on the app registration that trusts a specific repository and branch. Nothing to leak, nothing to rotate. This is the current recommended pattern and it's what I'd use if I built this properly.

I'd do it on a private repository first. My portfolio repos are public by design, which is the worst place to learn credential handling.